Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра **гуманитарных, математических и естественно научных дисциплин**

ПРАКТИЧЕСКАЯ РАБОТА

Дисциплина(модуль)/предмет ПМ 03 Сопровождение и обслуживание программного обеспечения компьютерных систем

Тема: Реализация версионирования API и проверка обратной совместимости в ASP.NET Core (предметный домен — каталог интернет-магазина)

ВАРИАНТ № —

Студент (ка)/учащийся: **Браунков Матвей Константинович** / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: **Чудинова А.А.** _____ /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «____» _____ 202__г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

Введение.....3
1 Постановка задачи и инструменты.....4
2 Листинг кода (Program.cs и контроллеры).....5
3 Результаты тестирования в Swagger UI.....7
Выводы.....9
Список использованных источников.....10

ВВЕДЕНИЕ

Цель работы — закрепить навыки проектирования параллельных версий REST API (v1 и v2) в экосистеме ASP.NET Core, освоить принцип аддитивных изменений (Additive Changes) и научиться защищать клиентские приложения от критических поломок (Breaking Changes). В качестве предметного домена выбран каталог интернет-магазина, для которого реализован сервис ApiVersioningLab.Braunkov на платформе .NET 8.0.

Поддержка нескольких версий API относится к процессам сопровождения и развития программного обеспечения по ГОСТ Р ИСО/МЭК 12207 [4]: она позволяет развивать сервис, не нарушая работу уже подключённых клиентов. В работе приводятся листинги кода, результаты тестирования в Swagger UI и выводы о том, почему аддитивное расширение версии v2 защищает клиентов от breaking changes.

1 Постановка задачи и инструменты

Необходимо создать веб-приложение ASP.NET Core Web API, поддерживающее две версии эндпоинтов — v1 и v2, где версия v2 расширяет v1 новыми полями без разрушения старого формата. Базовая версия v1 возвращает минимальный набор полей товара (id, title, price), а версия v2 — те же поля плюс аддитивные (currency, inStock, apiVersion).

Инструменты: Microsoft Visual Studio с поддержкой .NET 8.0, шаблон ASP.NET Core Web API на языке C#, библиотека управления версиями Asp.Versioning.Mvc [2] и генератор интерактивной документации Swagger (Swashbuckle) [3]. Проект назван ApiVersioningLab.Braunkov; маршруты версий различаются сегментом версии в URL (api/v1/products и api/v2/products) [1].

2 Листинг кода (Program.cs и контроллеры)

Настройка служб версионирования и генератора Swagger в файле Program.cs приведена в листинге 1.

Листинг 1 – Program.cs

```
using Asp.Versioning;
using Microsoft.OpenApi.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Регистрация служб версионирования API
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
})
.AddApiExplorer(options =>
{
    options.GroupNameFormat = "'v'VVV";        // имена групп: v1, v2
    options.SubstituteApiVersionInUrl = true;  // подстановка версии в URL
});

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1",
        new OpenApiInfo { Title = "Catalog API", Version = "v1" });
    options.SwaggerDoc("v2",
        new OpenApiInfo { Title = "Catalog API", Version = "v2" });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(options =>
    {
        options.SwaggerEndpoint("/swagger/v1/swagger.json", "Catalog API
v1");
        options.SwaggerEndpoint("/swagger/v2/swagger.json", "Catalog API
v2");
    });
}

app.MapControllers();
app.Run();
```

Контроллер версии v1 возвращает базовый набор полей со статусом 200 OK (листинг 2).

Листинг 2 – ProductsV1Controller.cs

```
using Asp.Versioning;
using Microsoft.AspNetCore.Mvc;
```

```
namespace ApiVersioningLab.Braunkov.Controllers;

[ApiController]
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/products")]
public class ProductsV1Controller : ControllerBase
{
    [HttpGet]
    public IActionResult Get()
    {
        var products = new[]
        {
            new { id = 1, title = "Ноутбук",  price = 75000 },
            new { id = 2, title = "Смартфон", price = 42000 }
        };
        return Ok(products);   // 200 OK
    }
}
```

Контроллер версии v2 сохраняет все поля v1 и добавляет новые аддитивные поля currency, inStock и apiVersion (листинг 3).

Листинг 3 – ProductsV2Controller.cs

```
using Asp.Versioning;
using Microsoft.AspNetCore.Mvc;

namespace ApiVersioningLab.Braunkov.Controllers;

[ApiController]
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/products")]
public class ProductsV2Controller : ControllerBase
{
    [HttpGet]
    public IActionResult Get()
    {
        var products = new[]
        {
            new {
                id = 1, title = "Ноутбук", price = 75000,
                currency = "RUB", inStock = true, apiVersion = "2.0"
            },
            new {
                id = 2, title = "Смартфон", price = 42000,
                currency = "RUB", inStock = false, apiVersion = "2.0"
            }
        };
        return Ok(products);   // 200 OK: старые поля + аддитивные
    }
}
```

3 Результаты тестирования в Swagger UI

После запуска приложения интерфейс Swagger UI по адресу /swagger корректно отображает обе версии API, переключаемые в списке «Select a definition». Для каждой версии выполнен запрос (Try it out → Execute); ответы сервера приведены на рисунках 1 и 2.

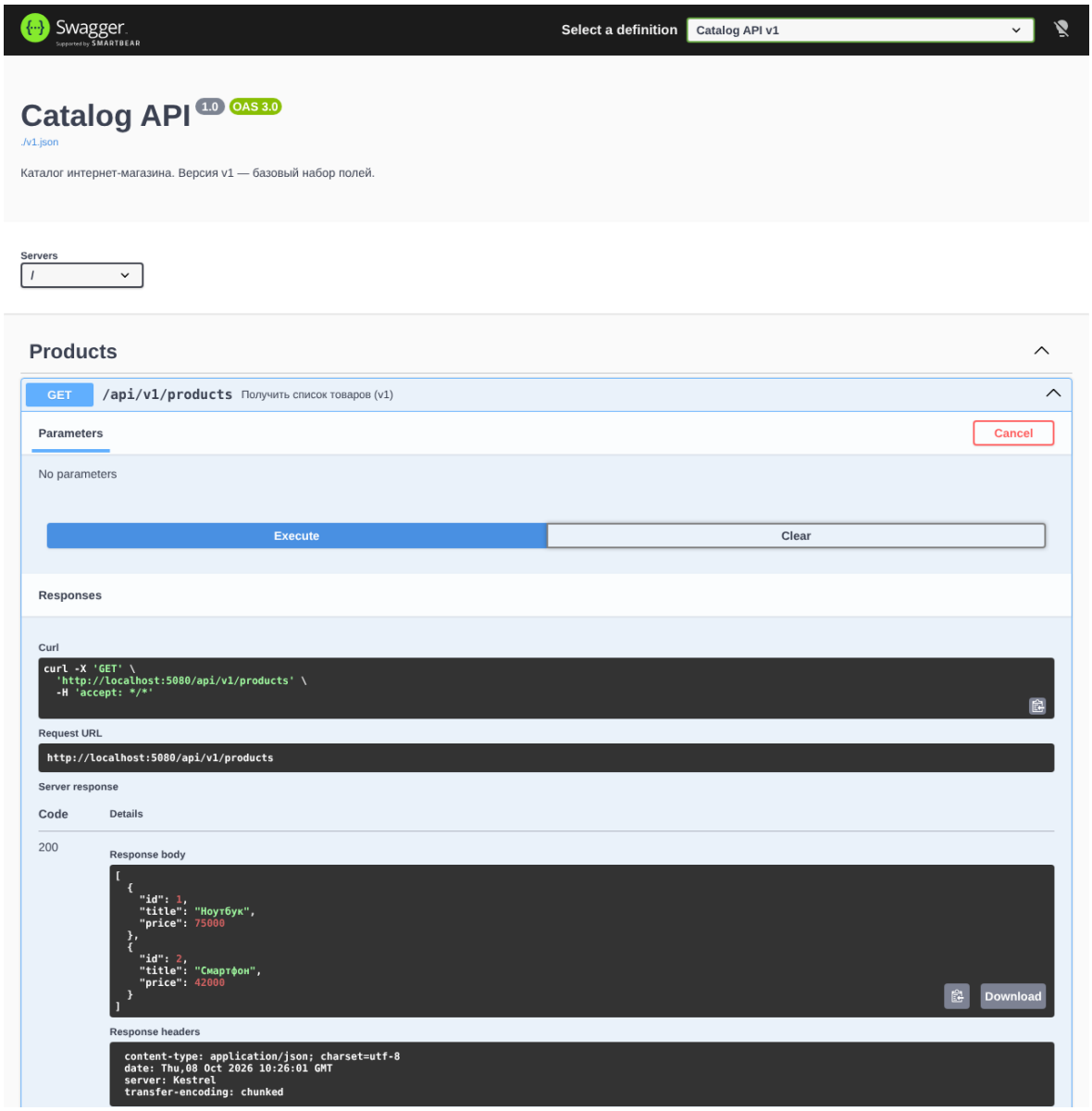


Рисунок 1 – Ответ сервера для версии v1 (поля id, title, price)

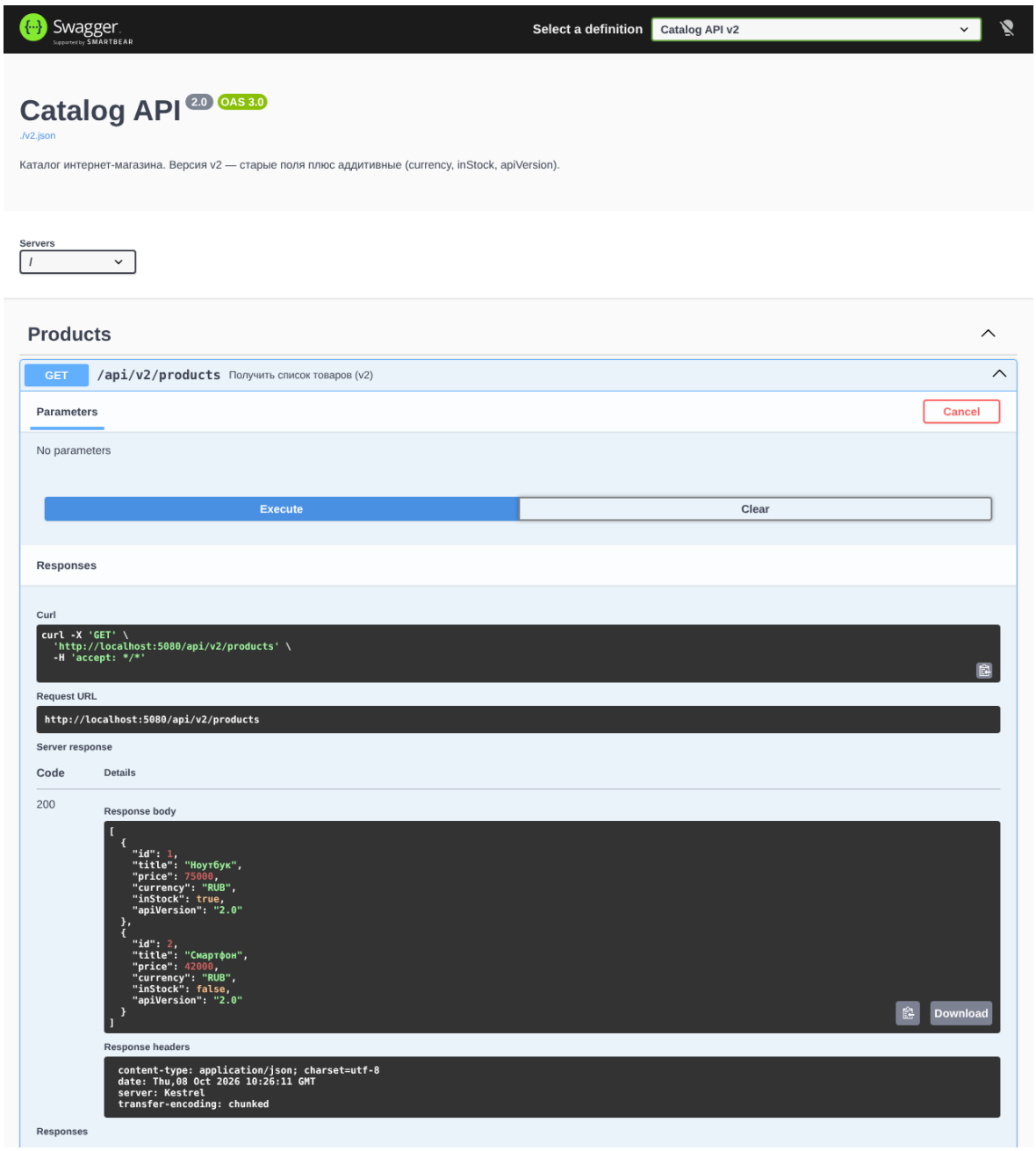


Рисунок 2 – Ответ сервера для версии v2 (добавлены currency, inStock, apiVersion)

Как видно из ответов, версия v2 содержит все поля версии v1 без изменения их имён и типов и дополняет их новыми полями; оба эндпоинта возвращают статус 200 OK.

ВЫВОДЫ

Добавление новых полей в версию v2 без изменения существующих структур — это аддитивное изменение, которое не ломает клиентов прежней версии. Старые клиенты продолжают читать знакомые поля (id, title, price) и просто игнорируют незнакомые им новые поля, поэтому их работа не нарушается. Параллельное существование эндпоинтов v1 и v2 позволяет переводить клиентов на новую версию постепенно, в удобные для них сроки. Тем самым соблюдается принцип обратной совместимости и исключаются breaking changes, что повышает надёжность сопровождения и развития сервиса [1], [2].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Создание веб-API в ASP.NET Core : документация Microsoft Learn. — Текст : электронный. — URL: https://learn.microsoft.com/ru-ru/aspnet/core/web-api (дата обращения: 08.10.2026).

2. API Versioning for ASP.NET Core (Asp.Versioning) : документация проекта. — Текст : электронный. — URL: https://github.com/dotnet/aspnet-api-versioning (дата обращения: 08.10.2026).

3. OpenAPI Specification 3.0 и Swagger UI : официальная документация. — Текст : электронный. — URL: https://swagger.io/specification (дата обращения: 08.10.2026).

4. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. — Москва : Стандартинформ, 2011. — 102 с.

5. Троелсен, Э. Язык программирования C# 10 и платформа .NET 6 / Э. Троелсен, Ф. Джепикс. — 11-е изд. — Санкт-Петербург : Диалектика, 2022. — 1264 с.